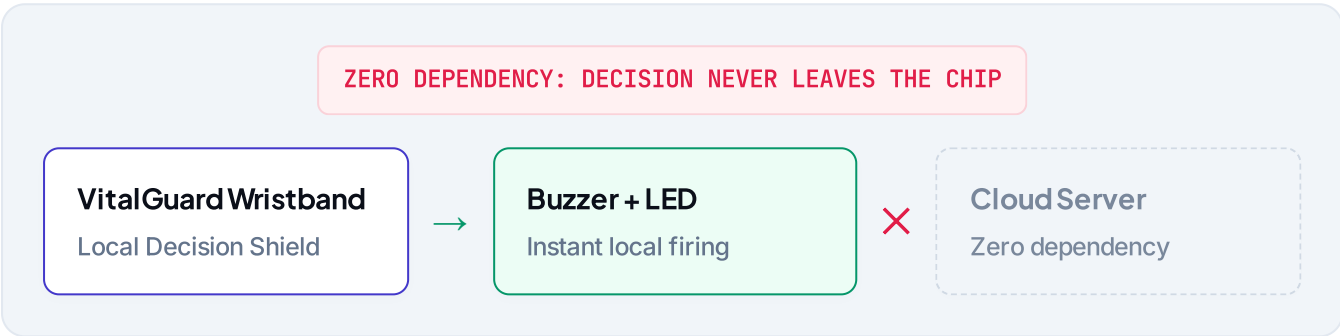


VitalGuard C3

An Edge-AI wrist-worn health monitor that senses, decides, and alerts entirely on-chip no internet, no paired phone, and no cloud server required for the safety decision itself.



Edge AI On-device inference	ESP32-C3 RISC-V microcontroller
1,578 Total hardware cost	Privacy-First Decision never leaves the chip

1. Problem Landscape & Electronica Context

The Electronica India Tech Challenge Brief

"Design an Edge AI powered wearable or embedded system that monitors physiological, environmental, or behavioral parameters and delivers real-time, on-device decision-making, without relying on the cloud or constant connectivity."

The Real-World Problem

Two pressures are compounding at once: chronic disease is rising, and the healthcare workforce is not growing fast enough to keep up. The two dominant monitoring approaches today each fail in a different way:

Periodic clinic visits

Health problems are only caught when the patient happens to be in front of a doctor. Between visits, a fall or a dangerous cardiac event goes unnoticed.

Cloud-connected wearables

Require a paired smartphone, stable internet, and often a subscription. Raw health data leaves the device, raising privacy concerns. If connectivity drops, so does protection.

Why Edge AI Changes This

Advances in ultra-low-power microcontrollers, intelligent sensors, and TinyML now allow the AI inference itself not just data collection to run locally on the wearable. The decision "is this person okay" is made on the wrist, in microseconds, whether or not a network exists nearby.

Who This Is Actually For

Elderly residents in old-age-care facilities, chronic-disease patients being monitored between clinic visits, and people in low-connectivity or rural settings specifically, people for whom a ₹15,000-40,000+ smartwatch with a monthly subscription is not realistic to deploy per person, and for whom simplicity (zero setup action required from the wearer) matters more than a feature-rich app.

2. Our Approach

VitalGuard C3 pairs two low-cost sensors with a single microcontroller that performs both sensing and decision-making locally. The core design principle: the safety-critical decision must never depend on connectivity, whatever else is added on top later.

Two Signals, Two Independent Decisions

Category	Sensor	What It Detects
Physiological	MAX30102	Heart rate and blood oxygen (SpO2), watched for irregular patterns
Behavioral	MPU6050	Motion signature of a fall free-fall dip, impact spike, post-fall stillness

These are kept as two separate alerts rather than one fused "diagnosis" a fall trigger and an irregular-heart trigger, each independently capable of firing the buzzer and LED.

Why Not Just Use an Existing Smartwatch

Commercial wearables already do on-device fall detection and heart monitoring that part isn't new. What they don't solve is reach: cost (₹15,000-40,000+), a required paired smartphone, and often a recurring subscription. None of that is viable for outfitting every resident in an under-staffed old-age home. VitalGuard C3's contribution isn't a new sensing technique it's making the same safety feature deployable at roughly 1,578 per unit, one-time cost, at facility scale.

Two-Phase Roadmap (Important Distinction)

PHASE1 CURRENT BUILD

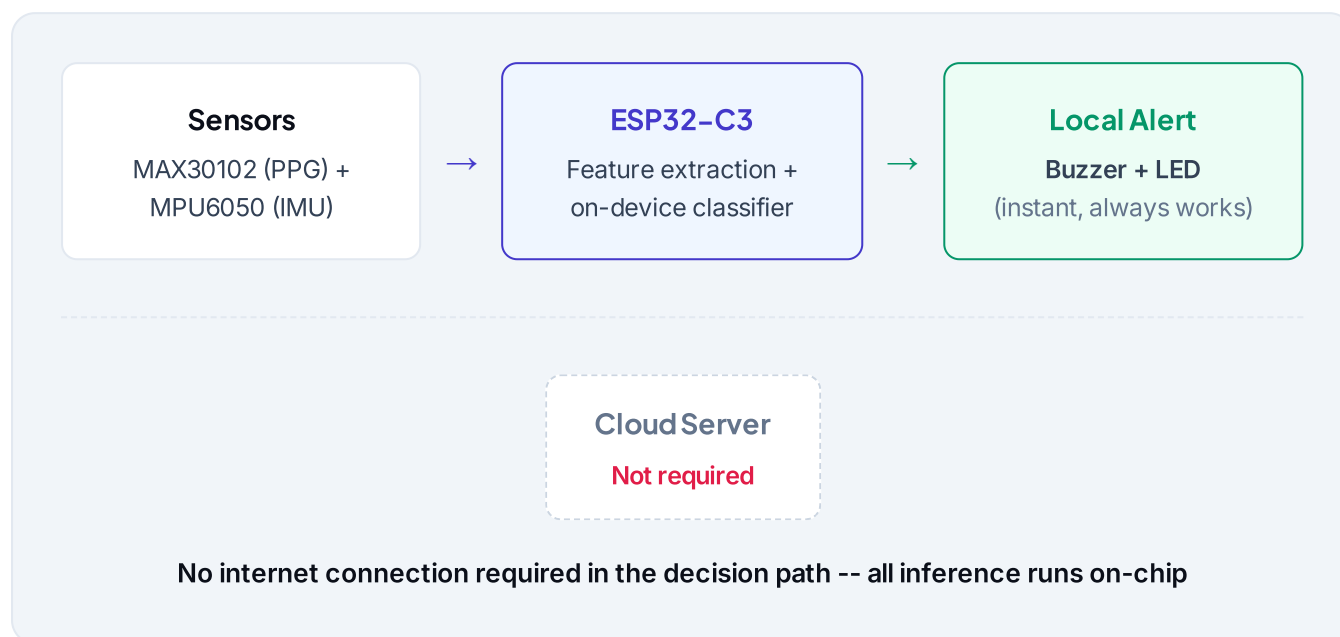
Fully edge-only. Sensing, feature extraction, and the trained classifier all run on the ESP32-C3. The buzzer/LED alert fires with zero network dependency. This is what is being submitted to the hackathon.

PHASE2 FUTURE ROADMAP

A companion dashboard application for facility managers and doctors, receiving vitals for record-keeping and trend review. Explicitly not part of the safety-decision path, and not part of the current hackathon submission see Section 9.

3. System Architecture

The current build's data flow, exactly as it will be demonstrated:



Sensors feed the ESP32-C3, which performs feature extraction and inference locally, and directly drives the local alert. No cloud server sits anywhere in this decision path the crossed-out connection above is deliberate, not a missing feature.

Design Rule Carried Through Every Layer

Any future feature (phone display, dashboard sync, doctor access) is only ever allowed to observe the decision after it has already been made on-chip. Nothing downstream is ever allowed to become a dependency for the alert itself.

4. Hardware & Sensor Engineering – Deep Dive

4.1 MAX30102 (Pulse Oximeter)

Working mechanism, in plain words: the sensor shines red and infrared light through the skin. Blood absorbs different amounts of each wavelength depending on how much oxygen it's carrying, and the amount reflected back changes slightly with every heartbeat. The chip measures this reflected light and converts it into two readings: heart rate (from how often the pattern repeats) and SpO₂, blood oxygen percentage (from the ratio of red to infrared absorption). Communicates over I²C (2 wires: SDA, SCL).

Accuracy stated honestly: wrist-based PPG is inherently noisier than a fingertip or a clinical pulse oximeter, because blood flow at the wrist is weaker. This is a physical limitation of the sensing method, not something firmware can fully correct. Readings are positioned as a wellness-grade indicator, not a diagnostic-grade measurement - no clinical validation has been performed.

Cost: 199 | Sourcing tier: hobbyist/maker-grade breakout board (Analog Devices/Maxim silicon lineage)

4.2 MPU6050 (6-Axis Accelerometer + Gyroscope)

Working mechanism, in plain words: contains microscopic mechanical structures (MEMS) that shift very slightly under acceleration or rotation. That physical shift changes a tiny electrical capacitance, which the chip reads and converts into six numbers: acceleration on X/Y/Z and rotation rate on X/Y/Z. Also communicates over I²C, sharing the same two wires as the MAX30102.

Accuracy stated honestly: fall-detection accuracy depends far more on the algorithm than the sensor. Published research shows a large gap between lab-simulated falls (often 90-99% claimed accuracy) and genuine real-world falls (averaging closer to 55-60% for simple threshold methods). Feature-engineered machine-learning approaches, the approach used here, have reached 80%+ sensitivity on real-world fall datasets in published research this project targets 70-80% real-world accuracy, in line with the best published results for this class of hardware, not an inflated claim.

Cost: 200 | Sourcing tier: hobbyist/maker-grade breakout board (TDK/InvenSense silicon lineage)

4.3-ESP32-C3 (Seeed XIAO)

Role: a 160MHz RISC-V microcontroller with integrated Wi-Fi and Bluetooth Low Energy, and 400KB of SRAM. Runs the feature-extraction math and the trained classifier directly on-chip,

and drives the alert output. Has built-in battery-charging support, so no separate charging circuit is needed.

Why this chip specifically: the classifier used here is a small decision-tree/random-forest model operating on a handful of engineered features not a large neural network so it comfortably fits and runs in microseconds on this hardware without needing a more powerful (and more expensive) chip.

Cost: 679 | Sourcing tier: Seeed Studio, Espressif RISC-V silicon

5. Bill of Materials & Cost Comparison

Component	Role	Cost ()
Seeed XIAO ESP32-C3	Microcontroller runs feature extraction and the on-device classifier	679
MAX30102	Heart rate + SpO2 sensor	199
MPU6050	6-axis motion sensor for fall detection	200
3.7V LiPo Battery (500mAh)	Power source, rechargeable via the board's built-in charging circuit	250
Buzzer + LED + Switch Parts	Local audio/visual alert	250
TOTAL HARDWARE COST		1,578

Why This Comparison Matters

VitalGuard C3
1,578 one-time cost.
No subscription. No required paired phone for the core alert.

Typical Commercial Wearable
15,000-40,000+ hardware cost.
Often requires a paired smartphone and/or a monthly subscription for advanced health features.

The point of this comparison is not that VitalGuard C3 is more advanced commercial wearables already do on-device fall/heart monitoring. The point is reach: this cost structure is what makes per-resident deployment across an under-funded facility realistic.

6. Software & Training Pipeline

There are two genuinely separate coding jobs here, using two different tools a common point of confusion worth stating clearly:

JOB 1 Python (laptop, done once)

Recorded sensor data is used to train a small classifier using scikit-learn or LightGBM. This is where the AI model actually learns the difference between a fall and normal movement, and between a normal and irregular heart pattern. The trained model is then converted into a C header file using micromigen, which compiles it into plain if/else logic.

JOB 2 Arduino IDE (runs on the chip, forever)

The Arduino IDE does not train anything - its role is to compile and flash the firmware that runs permanently on the ESP32-C3: reading the sensors, running the feature-extraction math, executing the already-trained classifier (the header file from Job 1), and driving the alert.

Libraries Used

Tool / Library	Purpose
Python 3, scikit-learn, LightGBM	Offline model training (Job 1)
micromigen	Converts the trained model into embedded C
Arduino IDE + Seeed ESP32-C3 board package	Firmware compilation and flashing (job 2)
SparkFun MAX3010x library	MAX30102 sensor driver
Adafruit MPU6050 library	MPU6050 sensor driver

7. Edge AI Firmware & Sensor Code

STATUS The code below shows the exact, real library calls used for sensor acquisition (Job 2) this part is standard and works as shown.

The classifier section shows the structure micromigen will generate once training data (Job 1) is collected that header file does not exist yet. Shown here to explain the design, not to claim a finished build.

7.1- Sensor Acquisition

Reads both sensors over the shared 120 bus

```
#include <Wire.h>
#include "MAX30105.N"
#include <Adafruit MPU6050.h>
MAX30105 pulseSensor,
Adafruit MPU6050 motionSensor;
void setup()
Wire.begin();
pulseSensor.begin(Wire, 12C SPEED FAST);
pulseSensor.setup();
motionSensor.begin();
```

7.2 Feature Extraction (runs every loop, on-chip)

```
// Matzom combine X/Y/Z into one how hard did it move" number
sensors event t accel, gyro, temp;
motionSensor.getEvent( accel, Sgyro, &temp);
float totalAccel = sqrt(
  accel.acceleration.x
  accel.acceleration.x
  accel.acceleration.y
  accel.acceleration.y
  accel.acceleration.z
  accel.acceleration.z
  this is the Signal Magnitude Vector (SKV), ng
// Heart: raw IR/Red reading feeds beat detection
long irValue pulseSensor.getIR();
Long redValue
pulseSensor.getRed();
HRV math (SparkFun Library)
```

7.3-On-Device Classifier (structure only not yet trained/generated)

```
// This header is auto-generated by micromigen AFTER Job 1 (Python training).  
// Shown here to illustrate the structure this file does not exist yet.  
#include "fall_model.h"  
Eloquent::ML::Port::DecisionTree fallClassifier:  
float features[] { minAccel, peakAccel, stillnessDuration:  
int result fallclassifier.predict(features):  
if (result FALL DETECTED) (  
triggerAlert("FALL");  
}
```

WORKED EXAMPLE Suppose totalAccel briefly drops to 0.3g (a free-fall dip), then spikes to 3.2g within a few hundred milliseconds (impact), then stays under 1.2g for the next 3 seconds (stillness). All three conditions together are what the classifier is trained to recognize as a fall. If a person simply sits down quickly, the impact spike may appear, but the free-fall dip and the stillness afterward typically won't both be present which is exactly why the three-stage pattern, not a single threshold, is used to keep false alarms down.

8. Honest Limitations & Mitigations

Presented here deliberately, not hidden a design that acknowledges its real constraints is more credible than one that claims none.

Accuracy

Real-world fall detection for this class of hardware realistically lands at 70-80% sensitivity, not the 90%+ often seen in lab-only testing. Heart rate/SpO2 from the wrist is inherently less accurate than a fingertip or clinical device a physical limitation, not a software bug.

Battery Life

With continuous active sensing (~25-40mA draw) and a 500mAh cell, realistic runtime is roughly 12-18 hours comparable to a smartwatch, not "days" or "weeks." Charging is via the board's built-in USB-C circuit, no extra hardware needed.

Unworn Detection (design, not yet built)

The MAX30102 produces a weak, flat signal when not in contact with skin, versus a clear pulsing waveform when worn. A sustained weak-signal condition can trigger a distinct "band not worn" alert important for cognitively impaired wearers who may not remember to re-wear it themselves.

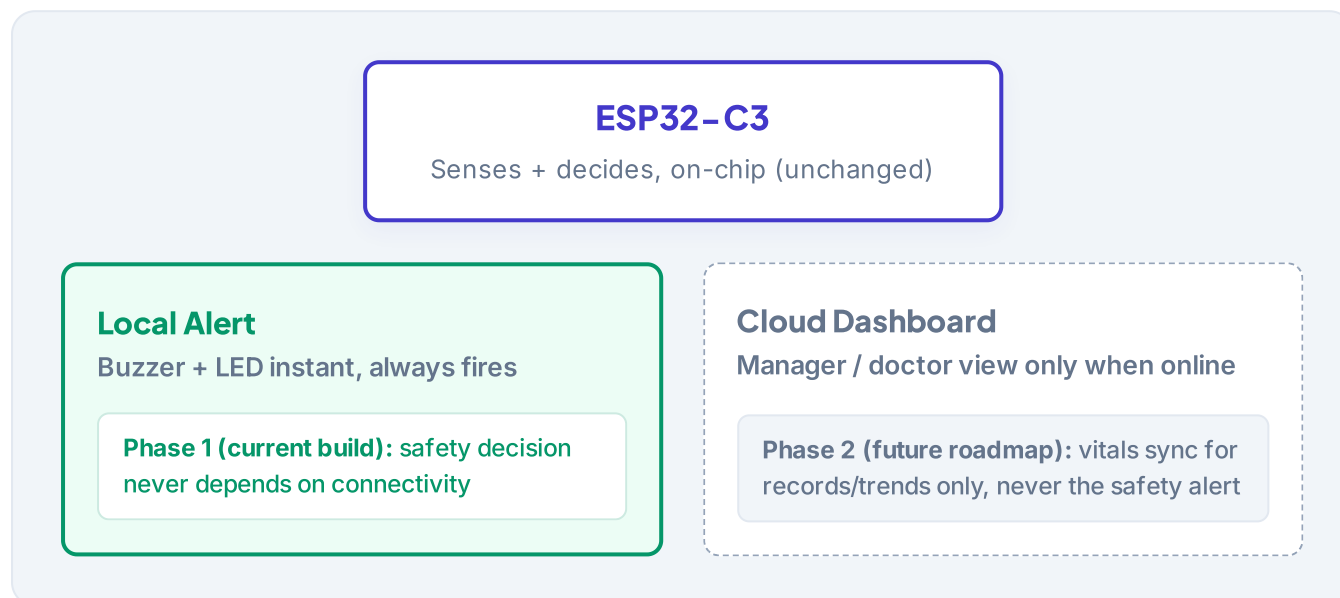
Dead Battery on an Unattended Wearer

If the battery dies while worn, monitoring stops this is a real limitation with no pure-firmware fix, and is true of every battery-powered wearable, including commercial ones. Mitigation is a combination of an early low-battery warning (around 20%, while still connected), longer runtime via duty-cycling in future firmware revisions, and a caregiver charging routine (e.g., during a resident's daily bath or meal time) a procedural safeguard, not a purely technical one.

9. Future Roadmap

Phase 2 (Not Part of Current Submission)

Once the edge-only core is proven, a companion layer can extend reach for facility managers and doctors explicitly built so it never becomes a dependency for the safety alert.



Planned Phase 2 Components

Stakeholder	What They'd Get
Facility Managers	Aggregate view of all residents' devices - battery status, last-seen, alert history
Attending Doctors	Longitudinal vitals history per patient for review between visits
End Users (residents)	Unchanged the wrist-worn alert continues to work exactly the same, with or without this layer

WHY THIS STAYS OUT OF THE CURRENT SUBMISSION The hackathon brief specifically asks for decision-making "without relying on the cloud or constant connectivity." Introducing a cloud dashboard into the core pitch risks diluting that exact compliance. This phase is presented as an honest post-hackathon product roadmap, kept clearly separate from what's being demonstrated now.

10. Summary

What VitalGuard C3 is: a 1,578 wrist-worn device that detects falls and irregular heart patterns entirely on-chip, alerting instantly with zero network dependency.

What it targets, honestly: 70-80% real-world fall-detection accuracy, wellness-grade (not diagnostic) vitals, 12-18 hours of runtime per charge.

What's built vs. planned: architecture and sensor design are finalized; firmware and model training are in progress; the Phase 2 dashboard is a future roadmap item, not part of this submission.

Prepared for the electronica India Tech Challenge 2026 - Messe München, presented by DigiKey BIEC, Bengaluru.